Szkriptnyelvek

Szathmáry László
Debreceni Egyetem
Informatikai Kar

8. Gyakorlat

- modulok
- random számok

(utolsó módosítás: 2026. jan. 16.)

2025-2026, 2. félév

Modulok

Amint a programunk egyre hosszabb lesz, felmerül az igény, hogy jó lenne szétvágni több részre.

Egyrészt könnyebb lenne a karbantartás, átláthatóbb lenne a projekt.

Másrészt egy-egy hasznos függvényt szeretnénk esetleg több programban is felhasználni anélkül, hogy a függvényt be kellene másolni minden egyes programba.

Feladat:

Írjunk két programot:

1.írassuk ki a 100-nál kisebb prímszámokat (`ex1.py`)

2.írassuk ki a 200-nál kisebb prímszámok összegét (`ex2.py`)

Link: https://arato.inf.unideb.hu/szathmary.laszlo/pmwiki/index.php?n=Py3.20121110a

ex1.py

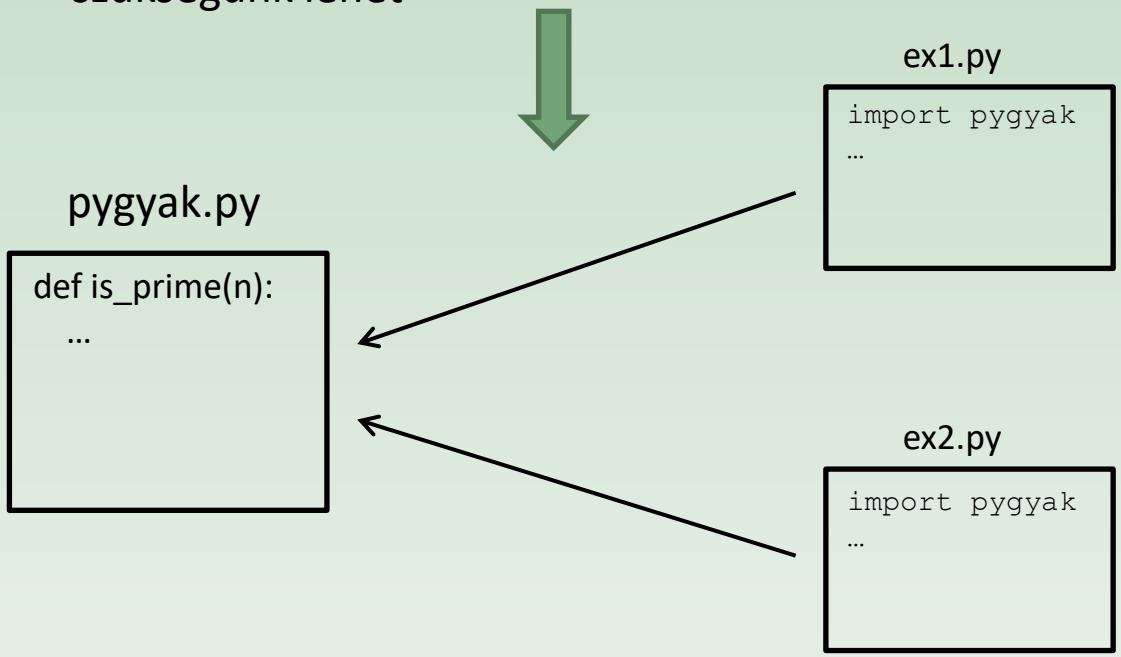
```
def is_prime(n):
    …
```

ex2.py

```
def is_prime(n):
    …
```

Miután készen vagyunk, nézzük meg, hogy mi a közös a két szkriptben. A közös rész az `is_prime()` függvény.

1. ugyanazt a függvényt ismételjük több helyen
2. az `is_prime()` olyan műveletet hajt végre, amire esetleg még máshol is szükségünk lehet



A közös függvényt tegyük ki egy modulba (pl. `pygyak.py`), majd mindkét szkriptben hivatkozzunk erre a modulra (importálás).

pygyak.py

```python
def is_prime(n):
    """
    Decide whether a number is prime or not.
    """
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3
    maxi = n**0.5 + 1
    while i <= maxi:
        if n % i == 0:
            return False
        i += 2

    return True


def hello():
    return "Hello, World!"
```

Használata (ex3.py):

```python
import pygyak


def main():
    print(pygyak.is_prime(5))

    print(pygyak.hello())

```

`import pygyak` # .py nélkül

Variációk

```
import pygyak
```

Ez a `pygyak`-ban definiált függvényeket nem teszi be az aktuális szimbólumtáblázatba. Csupán a modul neve, a `pygyak` kerül be a sz.-t.-ba. Ezért a modulon belüli függvényekre a modul nevén keresztül tudunk hivatkozni: `pygyak.is_prime(…)`

```
import pygyak as pgy
```

Ha a modul neve hosszú és/vagy sokszor hivatkozunk rá, akkor átnevezjetjük úgy, hogy teszünk rá egy aliast. Innentől: `pgy.is_prime(…)`

Variációk (folyt.)

```
from pygyak import is_prime
```

Jelentése: a `pygyak` modulból hozzuk be az `is_prime` függvény nevét a sz.-t.-ba.
!!! Ez a modul nevét NEM hozza be a szimbólumtáblázatba **!!!**
Pl. `print(pygyak.hello())` # hiba, a „pygyak” szimbólum ismeretlen

Megoldás:

1. `from pygyak import is_prime, hello`

2. `from pygyak import is_prime`
   `import pygyak`

3. `from pygyak import *`
   Nem ajánlott, átláthatatlan lesz hogy mit honnan importáltunk.

Modulok tesztelése

Előtte:

```python
def is_prime(n):
    """
    Decide whether a number
    is prime or not.
    """
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3
    maxi = n**0.5 + 1
    while i <= maxi:
        if n % i == 0:
            return False
        i += 2

    return True


def hello():
    """
    Classic example.
    """
    return "Hello, World!"
```

Utána:

```python
#!/usr/bin/env python3

"""
pygyak modul
============

A Python gyakorlathoz
keszitett modul.
A gyakran hasznalt fv.-eket
itt gyujtjuk ossze.
"""

def is_prime(n):
    """
    Decide whether a number
    is prime or not.
    """
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    i = 3
    maxi = n**0.5 + 1
    while i <= maxi:
        if n % i == 0:
            return False
        i += 2

    return True


def hello():
    """
    Classic example.
    """
    return "Hello, World!"

#######################

if __name__ == "__main__":
    for n in range(2, 20):
        if is_prime(n):
            print('{n} primszam'.format(n=n))
```

docstring

teszt

Modulok tesztelése (folyt.)

```
40
41  ########################
42
43  if __name__ == "__main__":
44      for n in range(2, 20):
45          if is_prime(n):
46              print('{n} primszam'.format(n=n))
```

tetszőleges tesztek

A feltétel (lásd 43. sor) <u>csak akkor igaz</u>, ha a modult közvetlenül futtatjuk a parancssorból (vagyis `./pygyak.py`). Ekkor lefut a teszt.

Ha <u>modulként</u> importáljuk, <u>akkor a feltétel hamis</u>, a teszt nem fut le.

Vagyis: ha megírunk egy modult, akkor azt nem csak modulként lehet használni, hanem önálló szkriptként is.

Feladat: Módosítsuk úgy a `pygyak.py` modult, hogy önálló indításkor kérjen be a felhasználótól egy egész számot, majd írja ki, hogy az prím-e vagy sem.
Ezután futtassuk le az `ex1.py` szkriptet ismét, amiben importáljuk ezt a módosított `pygyak.py` modult. Mit tapasztalunk?

Modulok tesztelése (folyt.)

pygyak.py

```python
#!/usr/bin/env python3


def hello():
    return "Hello, World!"

##########

if __name__ == "__main__":
    print(__name__)
```

ex1.py

```python
#!/usr/bin/env python3

import pygyak


def main():
    print(pygyak.hello())
    print(pygyak.__name__)

##########

if __name__ == "__main__":
    main()
```

```
$ ./pygyak.py
__main__
$
$ ./ex1.py
Hello, World!
pygyak
```

Modulok tesztelése (folyt.)

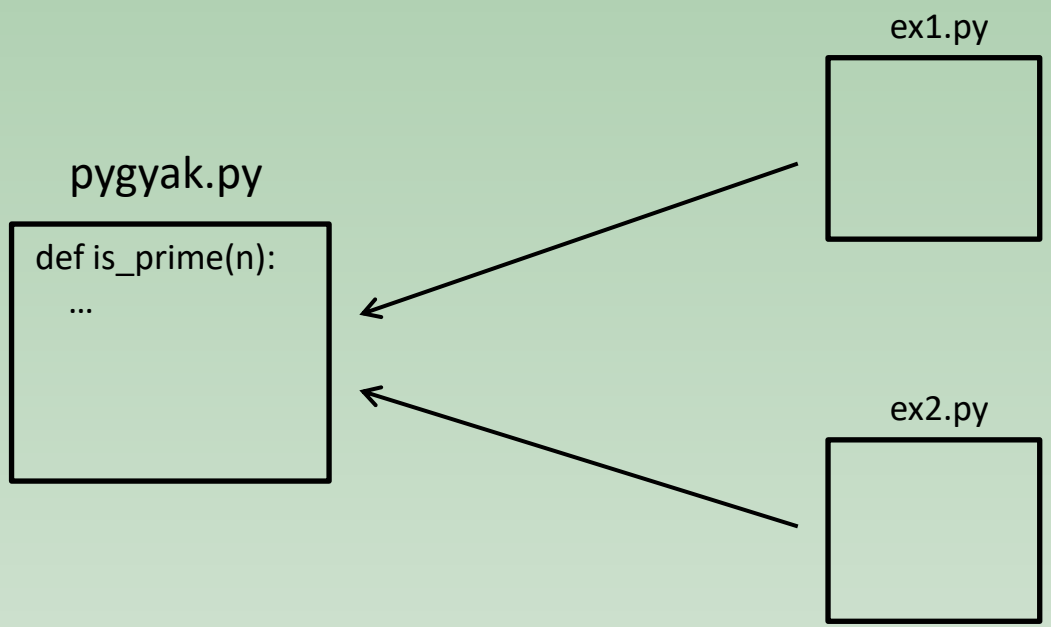
```
40
41  ########################
42
43  if __name__ == "__main__":
44      for n in range(2, 20):
45          if is_prime(n):
46              print('{n} primszam'.format(n=n))
```

További előny:

Ha a modulunk végére teszünk teszteket, akkor azzal a modult használó munkáját is segítjük, mivel konkrét példákat adunk a modul használatára.

„Alice: - Írtam egy király modult, elküldtem e-mailben. Megkaptad?
Bob: - Igen, de… Most akkor ezeket a függvényeket hogy kell használni?
Alice: - Menj le a forrás aljára, ott találsz pár példát.
Bob: - Ja, OK, látom már. Ehh, zsír. Köszi!"

Modulok



Modulok használatának további előnye

Ha egy függvényt módosítunk, akkor azt csak egyetlen helyen kell megtenni.

Feladat: A `pygyak.py` modulban cseréljük le az `is_prime()` függvény implementációját a jelenleginél jóval hatékonyabb Miller-Rabin tesztre.
Az MR teszt forráskódját a gyakorlati anyagok között találják.

Utána futtassuk le az `ex1.py` és `ex2.py` szkripteket. Mit tapasztalunk?

Modul importálásának a menete

```
import spam
```

Ezt szeretnénk importálni. Hogyan találja ezt meg az interpreter? Honnan fogja importálni?

1) Az interpreter megnézi, hogy van-e ilyen nevű beépített modul.
   Beépített modul: bele van fordítva az interpreterbe.

```
>>> import sys
>>> sys.builtin_module_names
```

2) Ha nem találta meg, akkor egy `spam.py` nevű fájlt fog keresni könyvtárak egy listájában, mely lista a `sys.path` -ban található.

```
>>> import sys
>>> sys.path
```

Ez a `sys.path` lista a következőképpen inicializálódik:

- a szkriptet tartalmazó könyvtár
- `PYTHONPATH` környezeti változó. Ez a `PATH` -hoz hasonló: ugyanúgy épül fel és könyvtárak listáját tartalmazza.
- az adott telepítésre jellemző alapértelmezett könyvtárak

Modul importálása (folyt.)

```
>>> import sys
>>> sys.path
```

A `sys.path` egy közönséges lista => az inicializáció után a saját szkriptünkből is módosíthatjuk.

!!! A futtatott szkriptet tartalmazó könyvtár a `sys.path` -ban *előre* kerül, a standard library mappái *elé*. Ez szokatlan hibákat okozhat, ha a könyvtár olyan fájlokat is tartalmaz, melyeknek a neve megegyezik valamelyik standard library-ben lévő modul nevével. !!!

„Lefordított” Python fájlok

Látható, hogy a `pygyak.py` mellett megjelent egy `pygyak.pyc` nevű file is. Ezt az interpreter automatikusan állította elő, ezzel nekünk nem nagyon kell foglalkozni.

Amikor egy modult importálunk, akkor az interpreter megpróbálja azt lefordítani, ezek lesznek a `.pyc` fájlok. Ez egy platformfüggetlen bináris kód.

Amikor importálunk egy modult és van belőle `.pyc` verzió, akkor az interpreter azt fogja betölteni. Az eredeti `.py` file módosítási ideje bele van fordítva a `.pyc` fájlba, vagyis ha a `.py` módosul, akkor az interpreter figyelmen kívül hagyja a `.pyc` fájlt (a `.py` fájlt fogja beolvasni és újrafordítja).

!!! Ha az interpreter a `.pyc` fájlt tölti be, attól még a programunk <u>nem fog gyorsabban futni</u>. **!!!** Csupán a modul betöltési ideje lesz gyorsabb.

Ha egy programot közvetlenül a parancssorból futtatunk, azt soha nem fogja az interpreter lefordítani. Csak az importált modulokat kísérli meg lefordítani.

Olyan is lehet, hogy csak a `spam.pyc` file van jelen a forrás nélkül. Ekkor az interpreter ezt fogja használni. Vagyis egy programot így is lehet terjeszteni, ha a forrást nem akarjuk kiadni. Ezt aránylag nehéz visszafejteni (reverse engineering).

Mi van egy modulban?

```
>>> import pygyak
>>> dir(pygyak)
['__builtins__', '__doc__', '__file__', '__name__', '__package__', 'hello', 'is_prime']
>>>
>>> pygyak.__doc__
'\npygyak modul\n============\n\nA Python gyakorlathoz \nkeszitett modul.\nA gyakran has
>>> print(pygyak.__doc__ )

pygyak modul
============

A Python gyakorlathoz
keszitett modul.
A gyakran hasznalt fv.-eket
itt gyujtjuk ossze.
```

a modul docstring -je

Random számok

https://docs.python.org/3/library/random.html

```
>>> import random
>>>
>>> random.random()
0.12214691572646652
>>> random.randint(1, 10)
9
>>> li = [3, 8, 2, 8, 4, 2, 1, 9]
>>> li
[3, 8, 2, 8, 4, 2, 1, 9]
>>> random.shuffle(li)
>>> li
[9, 8, 1, 4, 8, 2, 3, 2]
>>>
>>> random.choice(li)
8
```

`[0.0, 1.0)`

`lower <= N <= upper`

összekeveri az elemeket *helyben* (random permutáció)

random elem kiválasztása a listából

Feladat: a `shuffle()` eljárás *helyben* módosítja a listát. Írjunk egy olyan `shuffled()` nevű fv.-t, amely visszaadja az összekevert listát, lehetővé téve pl. a következőt: `n = shuffled(li)[-1]`

Feladat

Bullshit Generátor

Link: https://arato.inf.unideb.hu/szathmary.laszlo/pmwiki/index.php?n=Py3.20121030a

bullshit.py

bs_gen.py

```
import bullshit

def main():
    print(bullshit.get_bullshit())
```

Innen nézve a modul egy fekete doboz. Van egy interfésze, azon keresztül tudjuk használni, de az implementációja nem érdekel minket.

Feladatok

házi feladat

1. [20121110a] modulok
2. [20120905c] my shuffle
3. [20121030a] bullshit generátor
4. A **sor** és **verem** adatszerkezetek implementálása osztályok segítségével. Mindent dokumentáljunk le docstring-ekkel: a modult, az osztályt, az osztály függvényeit.